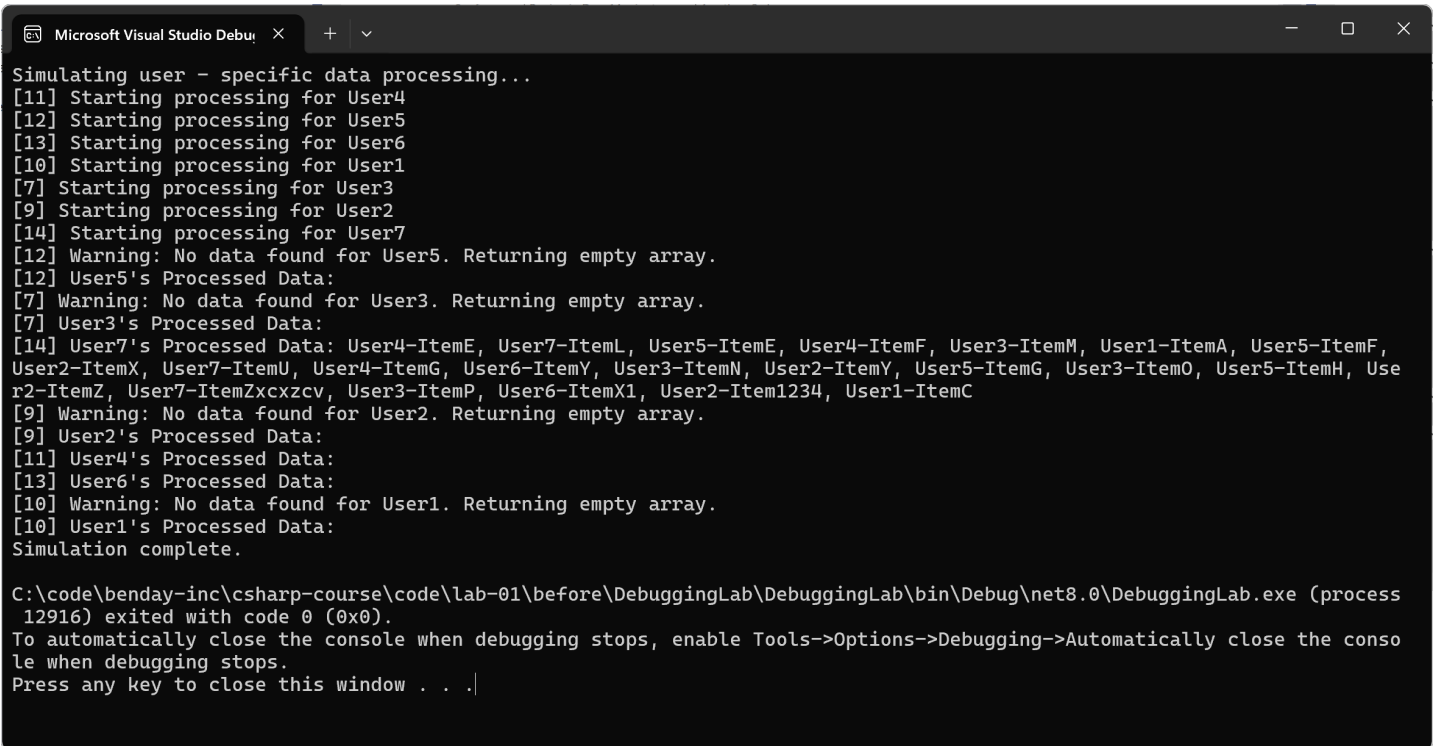


2. Observe the output. You should see:

- Both users' processed data includes items from both users.
- Example output:

```
Simulating user-specific data processing...
[4] Starting processing for User1
[5] Starting processing for User2
Initializing shared cache.
[4] User1's Processed Data: User1-ItemX, User1-ItemY
[5] User2's Processed Data: User1-ItemX, User1-ItemY
Simulation complete.
```

We're seeing data from both users when we should only be seeing output from a single user. **What's going on?**



```
Microsoft Visual Studio Debug Console
Simulating user - specific data processing...
[11] Starting processing for User4
[12] Starting processing for User5
[13] Starting processing for User6
[10] Starting processing for User1
[7] Starting processing for User3
[9] Starting processing for User2
[14] Starting processing for User7
[12] Warning: No data found for User5. Returning empty array.
[12] User5's Processed Data:
[7] Warning: No data found for User3. Returning empty array.
[7] User3's Processed Data:
[14] User7's Processed Data: User4-ItemE, User7-ItemL, User5-ItemE, User4-ItemF, User3-ItemM, User1-ItemA, User5-ItemF,
User2-ItemX, User7-ItemU, User4-ItemG, User6-ItemY, User3-ItemN, User2-ItemY, User5-ItemG, User3-ItemO, User5-ItemH, Use
r2-ItemZ, User7-ItemZxczcv, User3-ItemP, User6-ItemX1, User2-Item1234, User1-ItemC
[9] Warning: No data found for User2. Returning empty array.
[9] User2's Processed Data:
[11] User4's Processed Data:
[13] User6's Processed Data:
[10] Warning: No data found for User1. Returning empty array.
[10] User1's Processed Data:
Simulation complete.

C:\code\benday-inc\csharp-course\code\lab-01\before\DebuggingLab\DebuggingLab\bin\Debug\net8.0\DebuggingLab.exe (process
12916) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .|
```

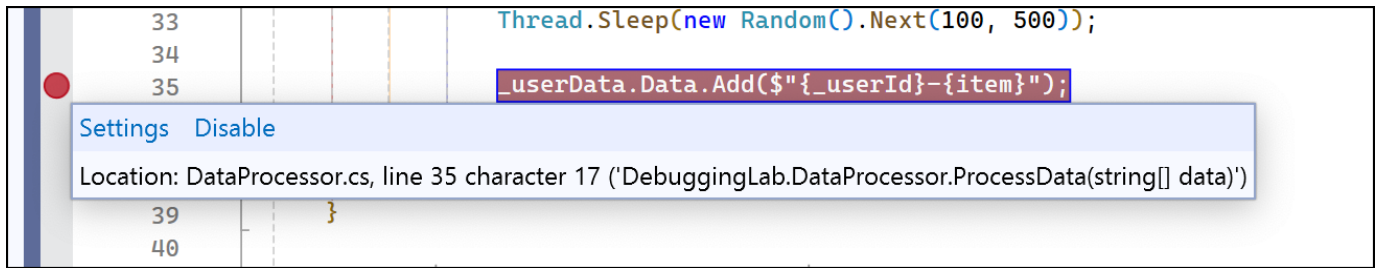
Step 3: Debugging with Breakpoints

Task 1: Line Breakpoint

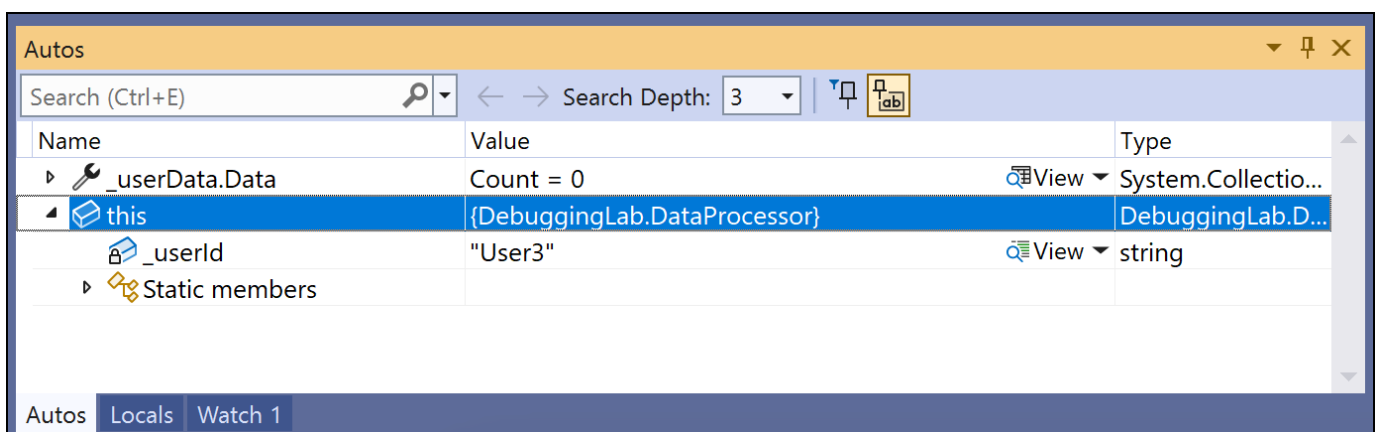
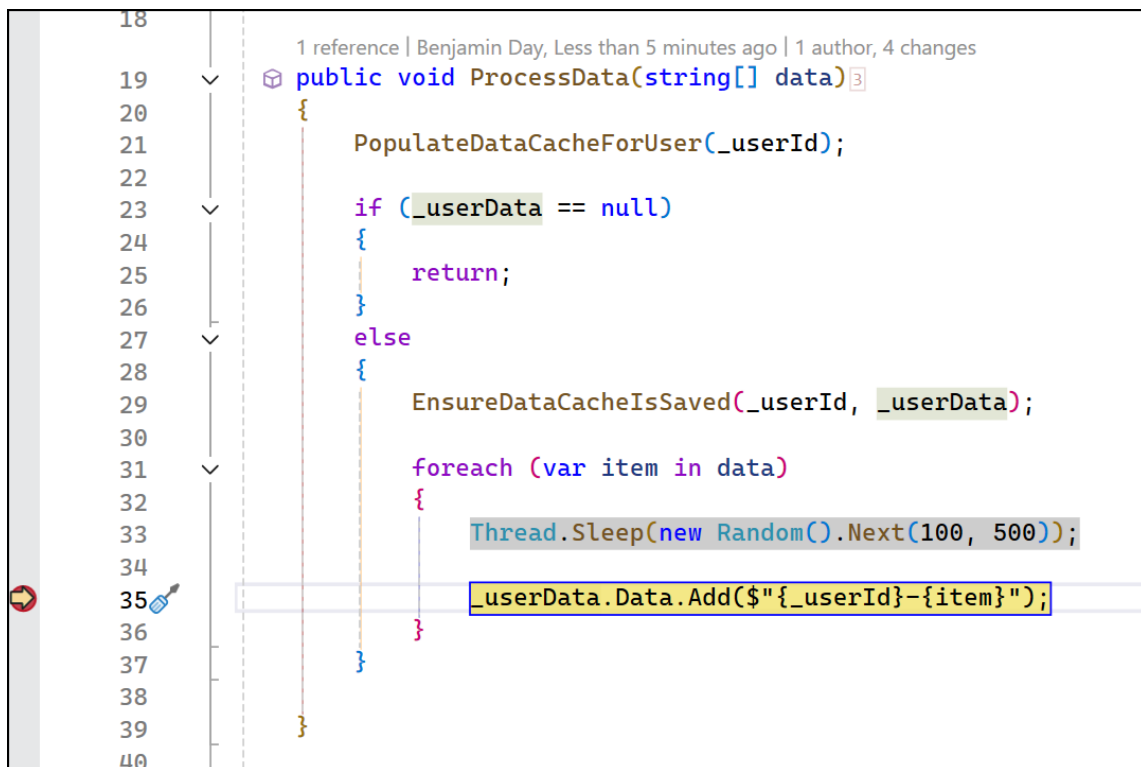
This is going to be a pretty basic demo of adding a breakpoint to a line and viewing the contents of variables using the debugger. Don't worry, we'll get to harder things in a bit.

1. In `DataProcessor.cs`, set a breakpoint on the following line in the `ProcessData` method by clicking in the left margin:

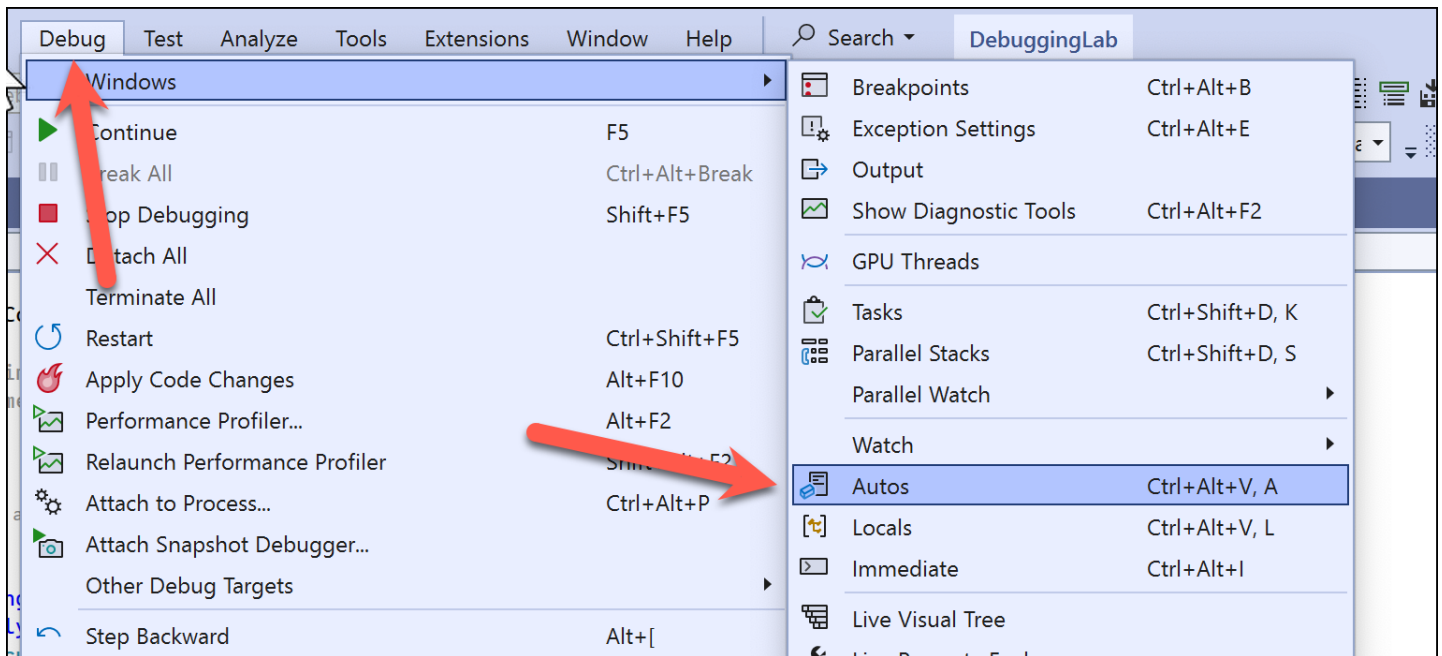
```
_userData.Data.Add($"{_userId}-{item}");
```



2. Run the program in debug mode (F5).
3. When the breakpoint hits, let's inspect the values in the Autos window for:
 - The value of `_userId`
 - The contents of `_userData.Data`



4. If the Autos window is not visible, you can open it by going to **Debug | Windows | Autos**:



5. Press **F5** to continue execution.

6. The breakpoint should get hit a another time...and again...and again...

Run Ignoring Breakpoints

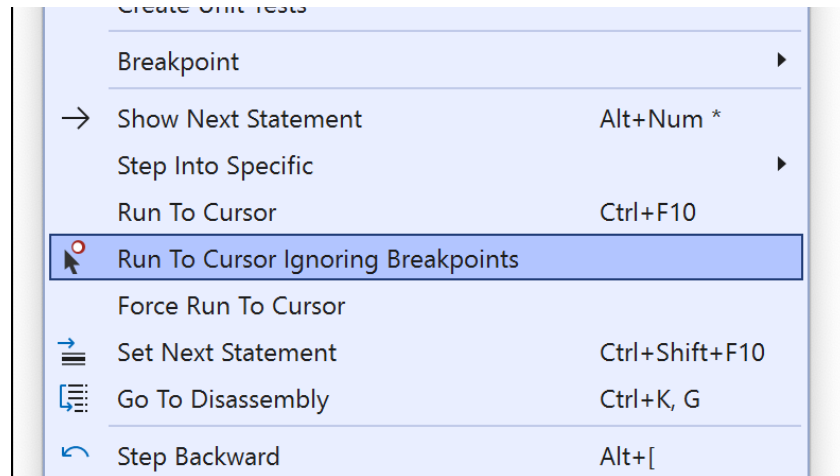
Let's assume that you've seen enough of this breakpoint. You could clear it - but sometimes when you're debugging, you'll want to use that breakpoint again but just skip ahead to a different part of the code. This is where **Run To Cursor Ignoring Breakpoints** can be helpful.

7. Open **Program.cs**

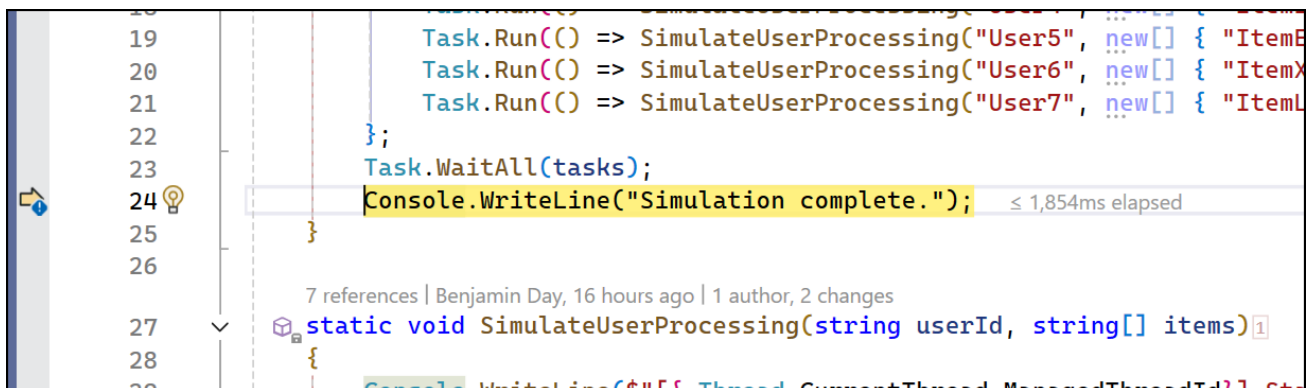
8. Click anywhere on **Line 24**. The goal is to place your cursor on this line

```
Console.WriteLine("Simulation complete.");
```

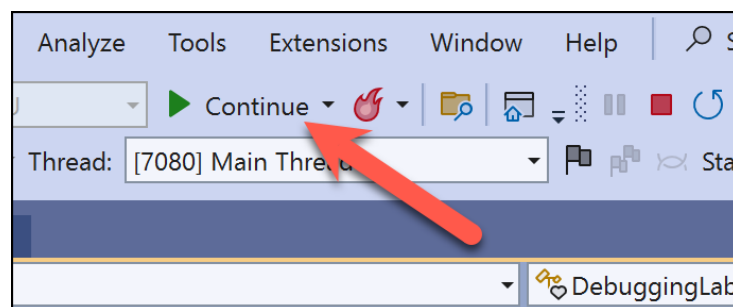
9. Right-click on **Line 24** and select **Run To Cursor Ignoring Breakpoints** from the context menu



10. Adding **Run To Cursor Ignoring Breakpoints** essentially adds a temporary line breakpoint. Execution should be paused and you should now be on **Line 24**



11. Press **F5** or click **Continue** to resume execution



Task 2: Conditional Breakpoint

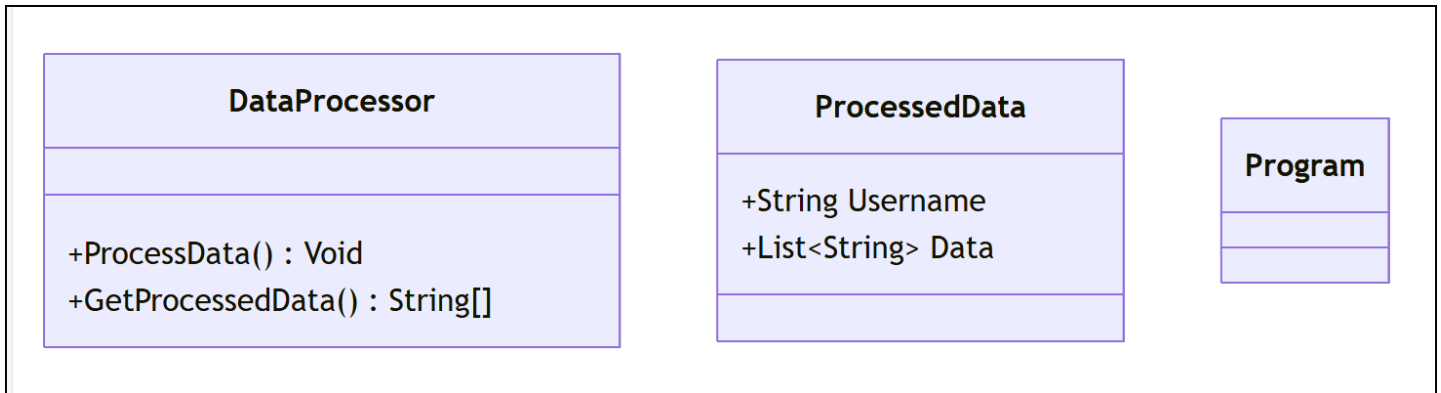
At this point, we haven't figured much out regarding our bug. It's still a mystery and the data is a jumble. But it's clear that processed data for users is being recorded to the wrong user account.

Conditional Breakpoints can be helpful for stopping when certain data scenarios occur. Think of this as a debugging hypothesis. In our case, it would be nice to know:

- Are we actually writing processed data to the wrong user account?

- If we are doing that, what is happening at that moment?

Below is a class diagram of the sample project. When **DataProcessor** is processing data for a user, it records the data that's been processed in the **ProcessedData** class.

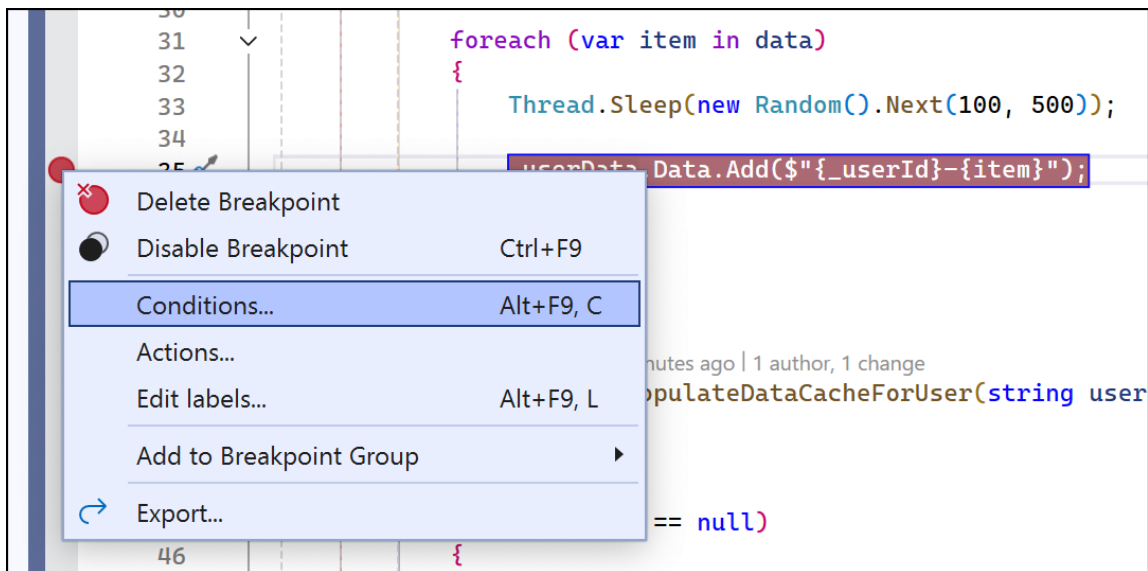


What we're going to do now is to modify the breakpoint so that it breaks to the debugger if the `ProcessedData.Username` doesn't match what we expect.

1. You should still have a breakpoint in **DataProcessor.cs** at **Line 35**.

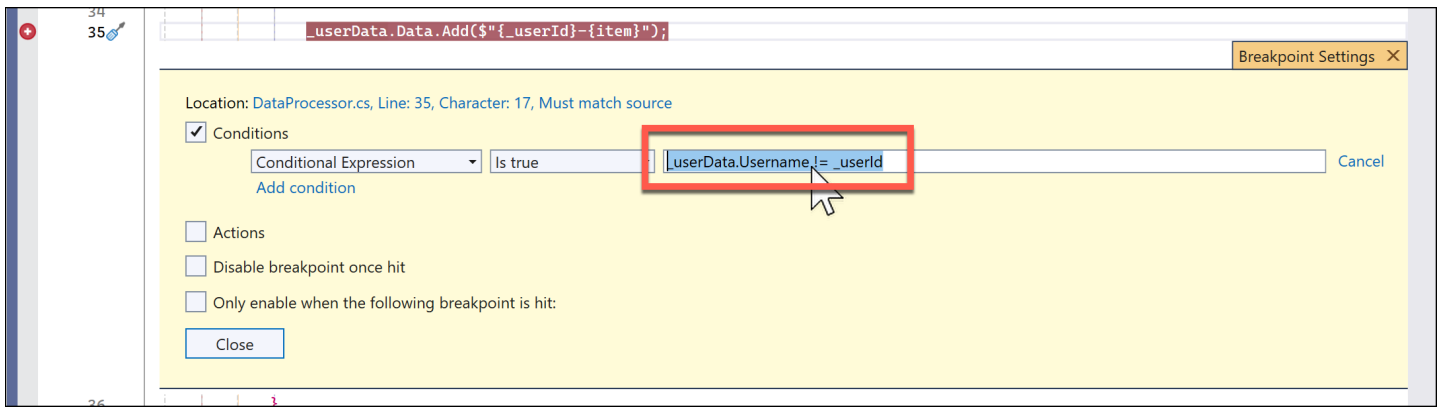
```
_userData.Data.Add($"{_userId}-{item}");
```

2. Change this breakpoint into a conditional breakpoint on the same line. To do this, **right-click** the breakpoint. From the context menu, choose **Conditions...**



3. You should now see a **Breakpoint Settings** dialog. Type the following text into the **Conditions** textbox and press **Enter**

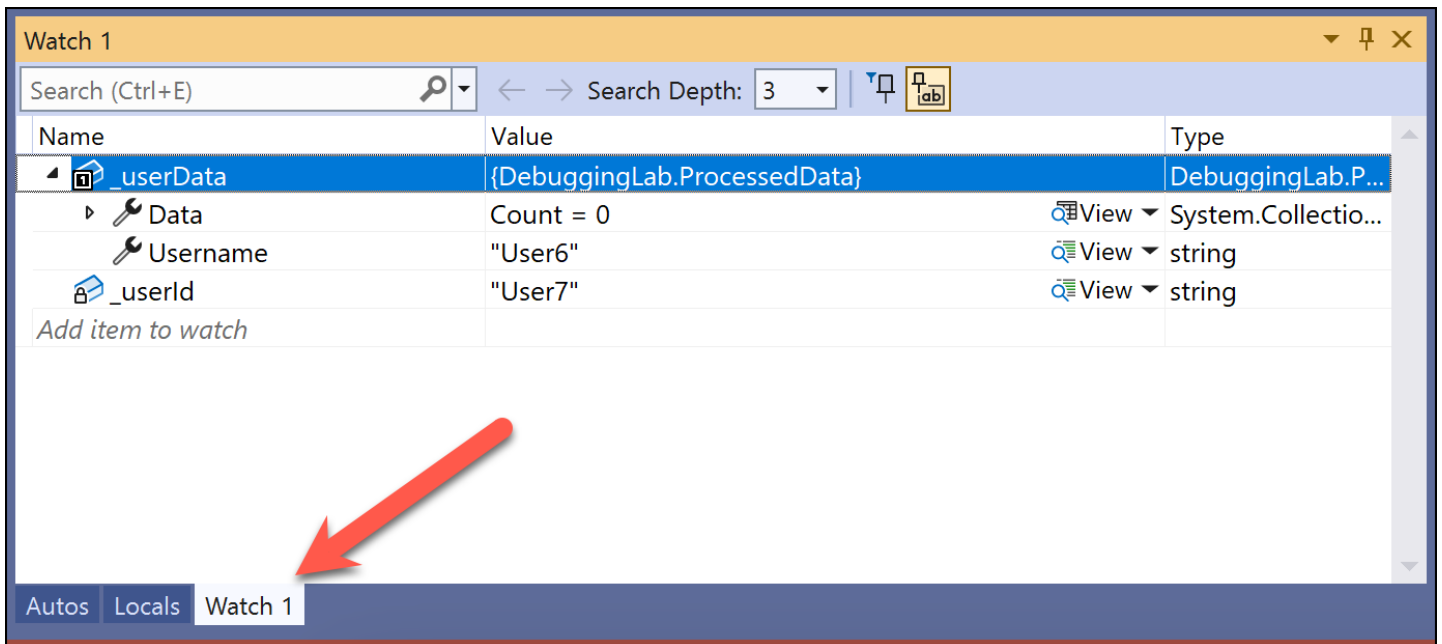
```
_userData.Username != _userId
```



4. Press **F5** to Run the program in debug mode.
5. The debugger should break at your conditional breakpoint

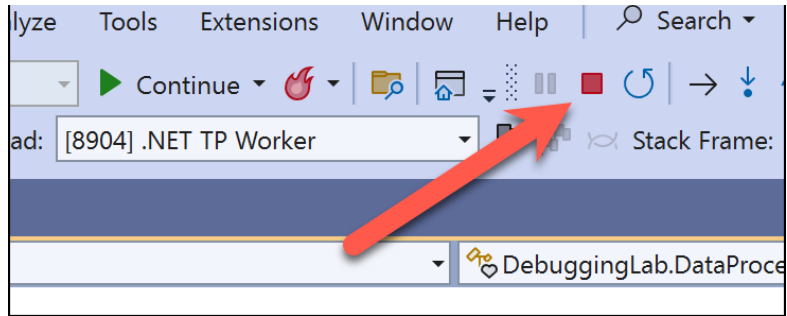
The **Autos** window and the **Locals** window for the debugger probably aren't showing you what you need to see all in one place. Specifically, I'd like to be able to see the current value of `_userId` and of `_userData.Username`.

6. Click the **Watch** tab in the debugger window.
7. Click where it says **Add item to watch** and add an item for both `_userId` and `userData`
8. Expand the entry for `_userData` so that you can see all the values. It should look something like the image below.



Notice how the username values do not match. That's not right.

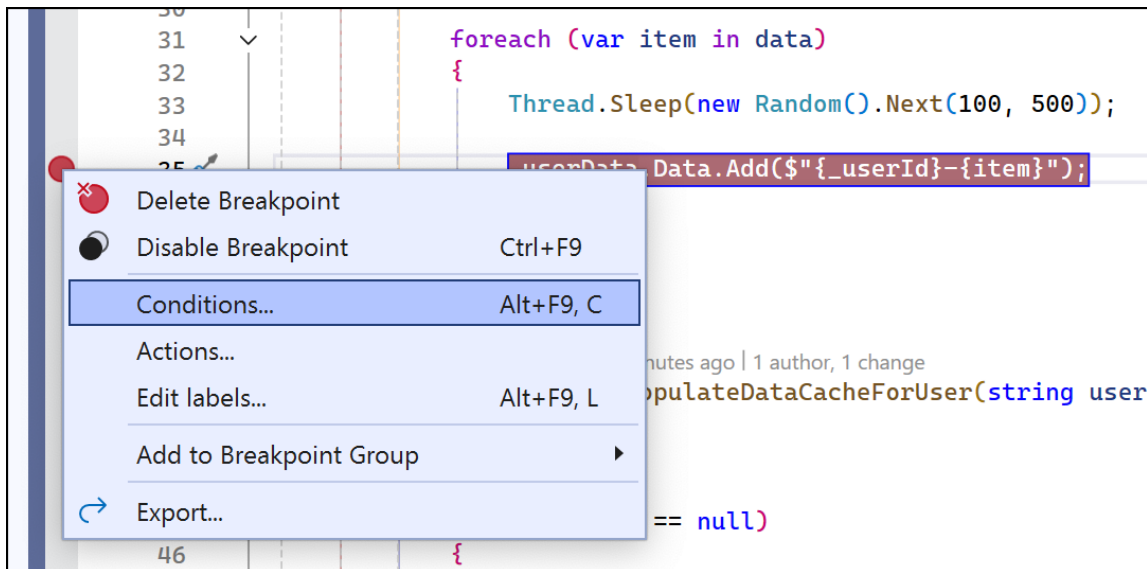
9. Click the **Stop** button to stop debugging



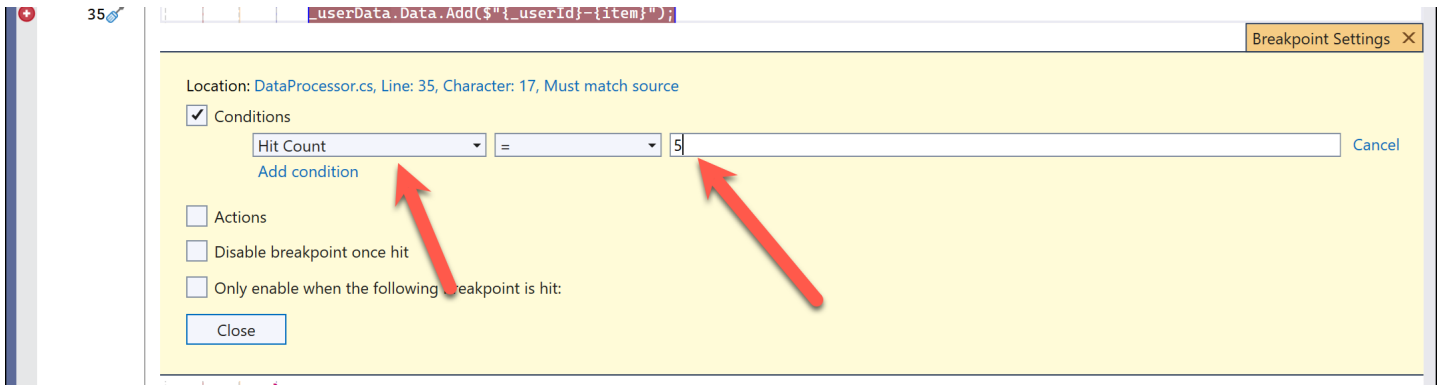
Task 3: Hit Count Breakpoint

When you're debugging code that's running in a loop, sometimes it can be helpful to stop after a certain number of iterations through the loop. You can do this using a **Hit Count** condition on a breakpoint

1. Remove the conditional breakpoint at **Line 35** in **DataProcessor.cs**
2. Add a new line breakpoint on the same line
3. Right-click the breakpoint > Choose **Conditions...**

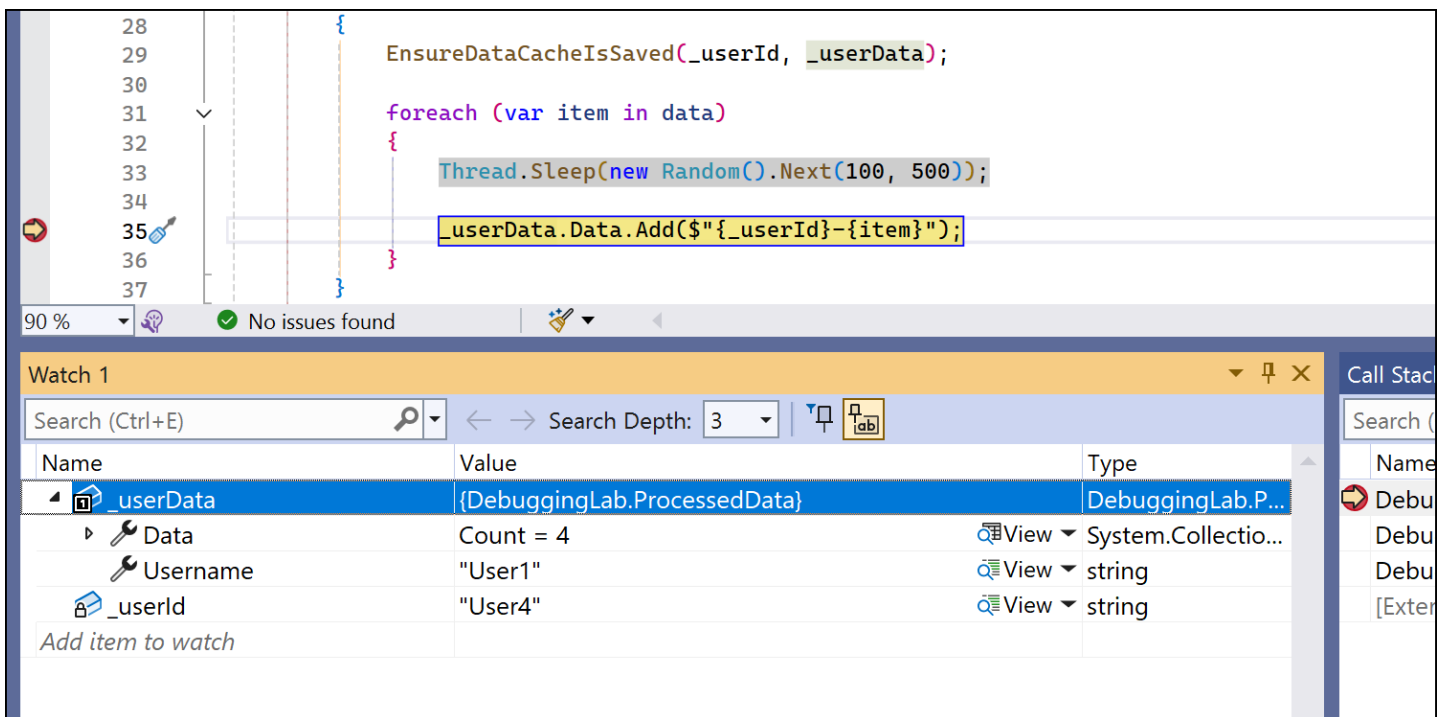


4. You should see the **Breakpoint Settings** editor. From the drop-down box, choose **Hit Count** then set the value to **5**. Then press **Enter**



4. Run the program in debug mode

5. You should hit the breakpoint and the username values probably don't match

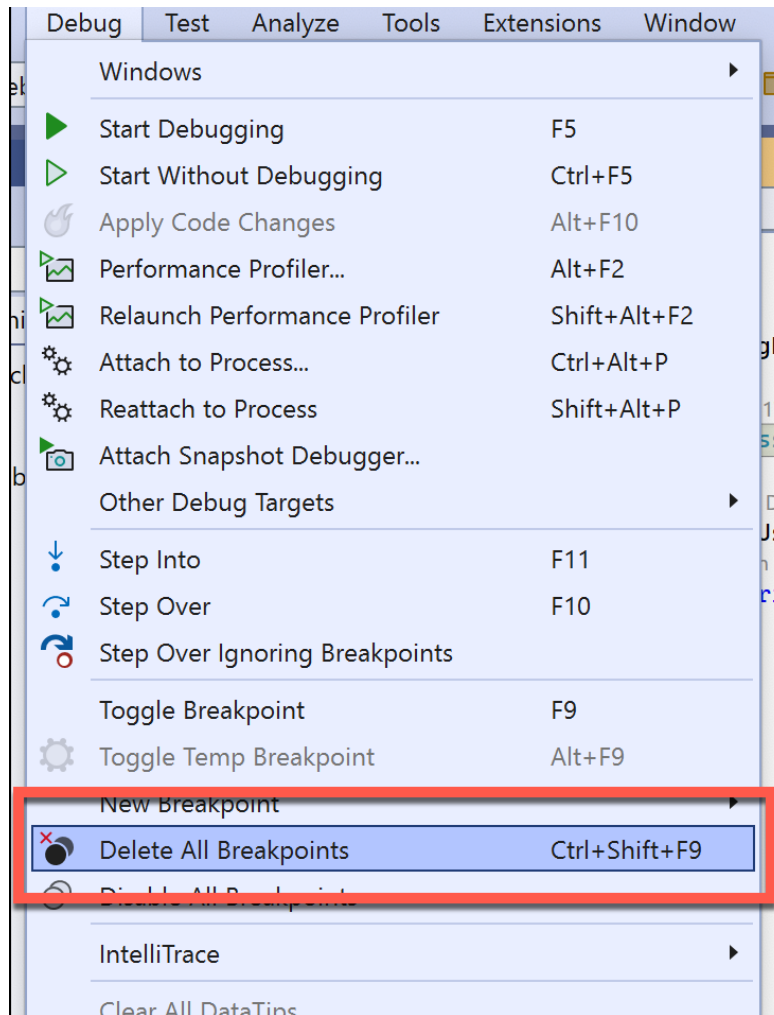


Yup. It's still broken. Let's keep digging.

Task 4: Delete All Breakpoints

Right now we only have one breakpoint but you can easily get into breakpoint sprawl. Helpfully enough, there's an option to delete all breakpoints.

1. From the **Main Menu** for Visual Studio, choose the **Debug** menu then choose **Delete All Breakpoints**

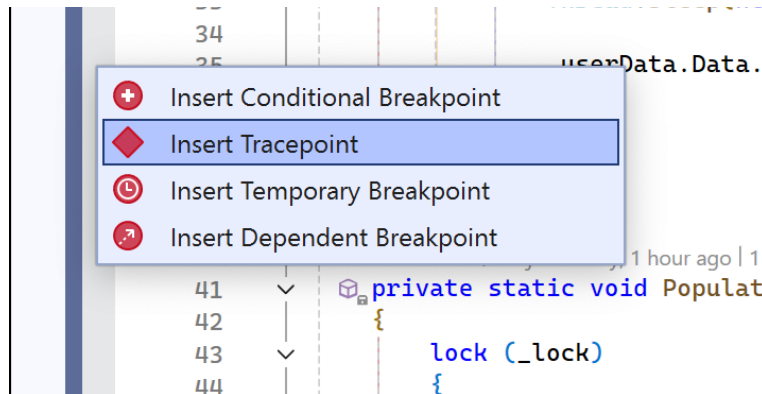


Task 5: Tracepoint

Sometimes you want to write a value to the console during execution rather than actually stopping execution and looking at the values in the debugger. You could do the time-honored practice of adding `Console.WriteLine()` or `Debug.WriteLine()` calls to your code. That works and it's easy to understand but -- well -- but that also leaves you with potentially lots of extra calls in your code.

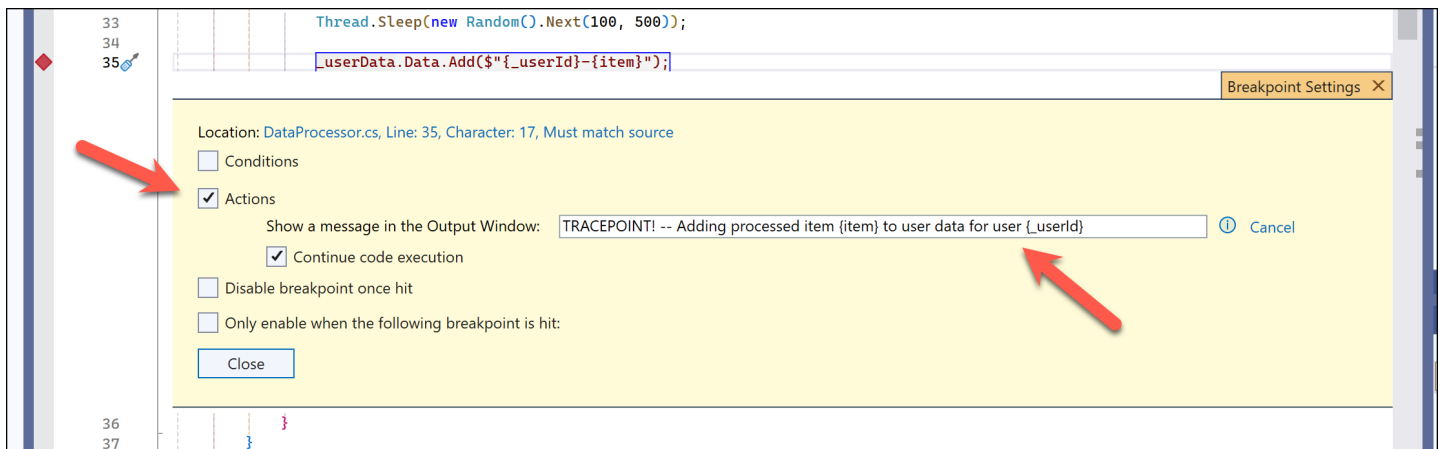
If you need a way to have the equivalent of a temporary `Console.WriteLine()`, you can use a **Tracepoint**.

1. Let's add a Tracepoint in **DataProcessor.cs** at **Line 35**. Go to that line and **right-click** in the left gutter. Choose **Insert Tracepoint** from the context menu



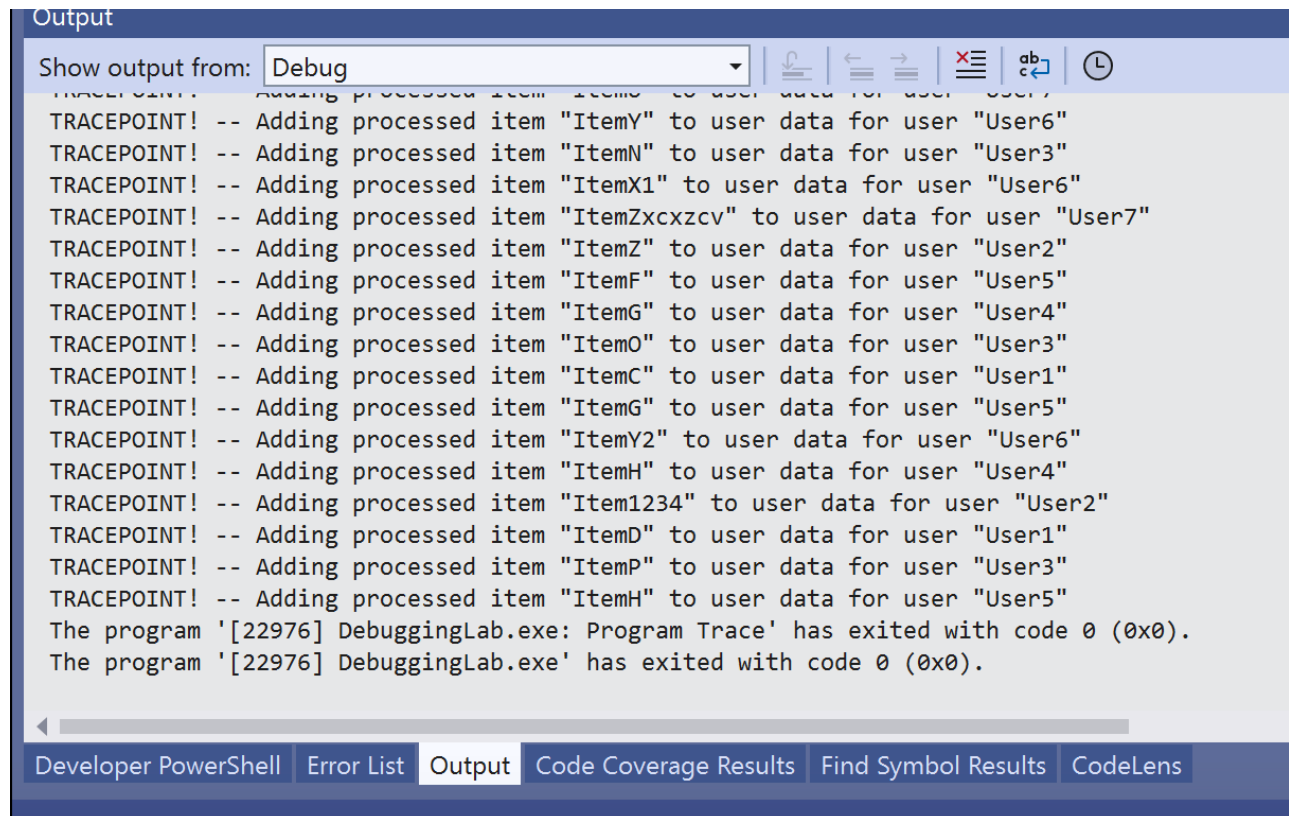
2. You should see the **Breakpoint Settings** dialog
3. Under **Actions** enter the following in the **Show a message in the Output Window** textbox

```
TRACEPOINT! -- Adding processed item {item} to user data for user {_userId}
```



4. Press **Enter**
5. Let's make sure that the **Output** window is visible. From the Visual Studio **Main Menu** choose **Debug > Windows > Output**
6. Press **F5** to run in debug mode
7. The application should run and complete.
8. Go to the **Output** window

You should see a bunch of tracepoint messages in the output.



Task 6 (Optional): Fix the Bug

I'm not sure if you've figured it out yet but this code is not great and whoever wrote it probably didn't understand `static` variables very well. If a variable is `static`, that means that there is a single instance that is shared and (potentially) accessible by all instances of the class. In this case, there was a combination of a race condition (a type of multithreading mistake) to initialize the `_userData` variable and then to make it worse, that `_userData` variable was marked as static.

All the changes are in **DataProcessor.cs**.

1. Change the `_userData` member variable to

```
private ProcessedData _userData;
```

2. Change the code for the constructor to

```
public DataProcessor(string userId)
{
    _userId = userId;
    _userData = new();
    EnsureDataCacheIsSaved(_userId, _userData);
}
```

3. Change the `ProcessData()` method to

```

public void ProcessData(string[] data)
{
    foreach (var item in data)
    {
        Thread.Sleep(new Random().Next(100, 500));

        _userData.Data.Add($"{_userId}-{item}");
    }
}

```

4. When you've completed the changes described above, the code for **DataProcessor** should look like the following code:

```

using System;
using System.Collections.Concurrent;
using System.Linq;

namespace DebuggingLab;

class DataProcessor
{
    private readonly string _userId;
    private static readonly ConcurrentDictionary<string, ProcessedData> _cache = new();
    private ProcessedData _userData;
    private static readonly object _lock = new();

    public DataProcessor(string userId)
    {
        _userId = userId;
        _userData = new();
        EnsureDataCacheIsSaved(_userId, _userData);
    }

    public void ProcessData(string[] data)
    {
        foreach (var item in data)
        {
            Thread.Sleep(new Random().Next(100, 500));

            _userData.Data.Add($"{_userId}-{item}");
        }
    }

    private static void EnsureDataCacheIsSaved(string userId, ProcessedData data)
    {
        if (_cache.ContainsKey(userId) == false)
        {
            cache.TrvAdd(userId, data):

```

```

    }
}

public string[] GetProcessedData()
{
    lock (_lock)
    {
        if (_cache.TryGetValue(_userId, out var processedData))
        {
            return processedData.Data.ToArray();
        }
        Console.WriteLine($"[{Thread.CurrentThread.ManagedThreadId}] Warning: No data found for {_userId}. Returning empty array.");
        return Array.Empty<string>();
    }
}
}

```

5. Press **F5** to run the application
6. You should see output similar to the screenshot below and all the user data should match the appropriate username.

```

Microsoft Visual Studio Debug
Simulating user - specific data processing...
[9] Starting processing for User3
[11] Starting processing for User4
[13] Starting processing for User6
[12] Starting processing for User5
[7] Starting processing for User2
[10] Starting processing for User1
[14] Starting processing for User7
[13] User6's Processed Data: User6-ItemX, User6-ItemY, User6-ItemX1, User6-ItemY2
[14] User7's Processed Data: User7-ItemL, User7-ItemU, User7-ItemZxcxzc
[7] User2's Processed Data: User2-ItemX, User2-ItemY, User2-ItemZ, User2-Item1234
[10] User1's Processed Data: User1-ItemA, User1-ItemB, User1-ItemC, User1-ItemD
[11] User4's Processed Data: User4-ItemE, User4-ItemF, User4-ItemG, User4-ItemH
[9] User3's Processed Data: User3-ItemM, User3-ItemN, User3-ItemO, User3-ItemP
[12] User5's Processed Data: User5-ItemE, User5-ItemF, User5-ItemG, User5-ItemH
Simulation complete.

C:\code\benday-inc\csharp-course\code\lab-01\before\DebuggingLab\DebuggingLab\bin\Debug\net8.0\DebuggingLab.exe (process 24544) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .

```

Fixed it!

